

SCRUM

Lecture 05

1

Scrum Framework

- Scrum is a simple lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.
- The basic practices and mechanics of Scrum are straightforward, and not difficult to adopt.
- But understanding how Scrum's values can make those practices and mechanics result in better software can be more challenging.

Agile Software Development

2

2

Basic Scrum Pattern

- There are three main roles on a Scrum project: **Product Owner**, **Scrum Master**, and **team member**.
- Product Owner works with the rest of the team to maintain and prioritize a **product backlog** of features and requirements that need to be built.

Agile Software Development

3

3

Basic Scrum Pattern

- Software is built using timeboxed iterations called **sprints**.
- At the start of each sprint, the team does **sprint planning** to determine which features from the backlog they will build.
- This is called **the sprint backlog**, and the team works throughout the sprint to build all of the features in it.

Agile Software Development

4

4

Basic Scrum Pattern

- Every day, the team holds a short face-to-face meeting called the **Daily Scrum** to update each other on the progress they've made, and to discuss the roadblocks ahead.
- Each person answers three questions:
 1. What have I done since the last Daily Scrum?
 2. What will I do until the next Daily Scrum?
 3. What roadblocks are in my way?

Agile Software Development

5

5

Basic Scrum Pattern

- Scrum Master, keeps the project rolling by working with the team to get past roadblocks that they've identified and asked for help with.
- At the end of the sprint, working software is demonstrated to the product owner and stakeholders in the **sprint review**.
- Team holds a **retrospective** to figure out lessons they've learned, so they can improve the way they run their sprints and build software in the future.

Agile Software Development

6

6

Basic Scrum Pattern



Agile Software Development

7

7

Scrum & Self-Organizing Teams

- But for a Scrum team to become effective, they need to do more than just follow the basic Scrum pattern.
- Effective Scrum teams are **self-organizing**.
- For Scrum to work, the team has to deeply and viscerally understand **collective commitment** and **self-organization**.
- Scrum's theory, practices, and rules are easy to grasp intellectually.

Agile Software Development

8

8

Scrum & Self-Organizing Teams

- But until a group of individuals has made a collective commitment to deliver something tangible in a fixed amount of time, those individuals probably don't get Scrum.
- When the team members stop acting as many and adopt and commit to a common purpose, the team becomes capable of self-organization and can quickly cut through complexity and produce actionable plans.

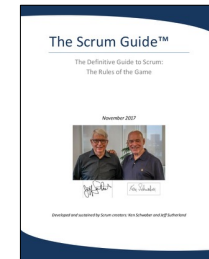
Agile Software Development

9

9

The Scrum Guide

- Developed and sustained by Scrum creators: Ken Schwaber and Jeff Sutherland.



Agile Software Development

10

10

Rules of Scrum

- Each sprint starts with **Sprint planning** meeting.
- Scrum Master, Product Owner, and the rest of the team is involved.
- Meeting is divided into two parts.
- At first part, Product Owner works with the team to select items that will be delivered at the end of the sprint.

Agile Software Development

11

11

Rules of Scrum

- Team agrees to give a demo of working software that includes those items at the end of the sprint.
- Second part of the meeting, team members figure out the individual tasks they'll use to actually implement those items.
- At the end of sprint planning, the items they've selected become the **sprint backlog**.

Agile Software Development

12

12

Rules of Scrum

- Team holds a **Daily Scrum** meeting every day.
- All team members must attend.
- Every team member answers 3 questions:
 1. What have I done since the last Daily Scrum?
 2. What will I do between now and the next Daily Scrum?
 3. What obstacles and roadblocks are in my way?

Agile Software Development

13

13

Rules of Scrum

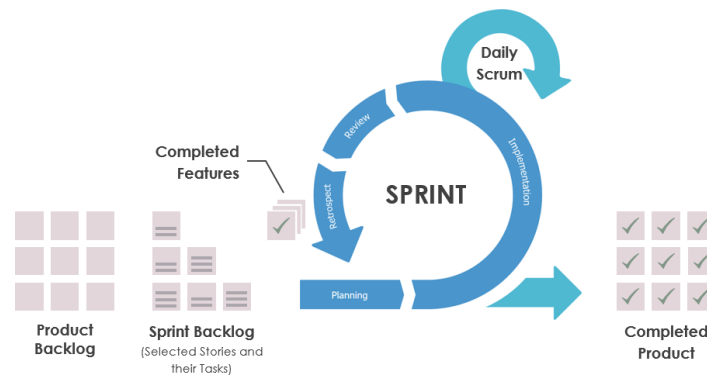
- Each sprint is timeboxed to a specific length decided during sprint planning.
- Length can vary.
- Some teams choose two-week sprints, some choose one month.
- During the sprint, the team builds the items in the sprint backlog into working software.

Agile Software Development

14

14

Rules of Scrum



Agile Software Development

15

15

Rules of Scrum

- Product owner is the team member who can work with the users and stakeholders to reset their expectations.
- He can adjust the sprint backlog to match the team's actual capacity.
- Team must keep the sprint backlog up to date and visible to everyone.

Agile Software Development

16

16

Rules of Scrum

- At the end of the sprint, the team holds a **sprint review** meeting.
- They demonstrate working software to users and stakeholders.
- At the end of the demo, the stakeholders are asked for their opinions and feedback.
- If changes are needed, that's taken into account when the next sprint is planned.

Agile Software Development

17

17

Rules of Scrum

- After the sprint, the team holds a **sprint retrospective** meeting.
- Finds specific ways to improve how they work.
- Each person answers two questions:
 - What went well during the sprint?
 - What can improve in the future?
- Scrum Master takes note of any improvements.

Agile Software Development

18

18

Everyone Owns the Project

- Every Scrum project has a Product Owner, a Scrum Master, and a team.
- But not every project that has those roles assigned to people is an effective Scrum project.
- When the Scrum Master, Product Owner, and team start to work together instead of separately, the project begins to look more like Scrum.

Agile Software Development

19

19

Everyone Owns the Project

- On effective Scrum teams, the team members don't just commit to their individual tasks.
- Every single person feels a genuine commitment to delivering the most valuable software they can to their users and stakeholders.
- When everyone on the team shares this feeling, and agrees to deliver working software at the end of each sprint, we say that team has made a **collective commitment**.

Agile Software Development

20

20

Scrum Has Its Own Set of Values

- Every company has its own culture that includes specific values.
- Every methodology has values built into it.
- Same goes for any value or principle: if it clashes with the company's values, it presents a barrier to adoption.
- In a company where the culture matches the agile values and principles, an agile team will be much more successful than a command-and-control team.

Agile Software Development

21

21

Scrum Has Its Own Set of Values

- Self-organizing teams work differently than command-and-control teams because they have different values
- There are five Scrum values: **commitment, respect, focus, openness, and courage.**
- Understanding self-organization starts with learning how these values are practical things that can be incorporated into your projects.

Agile Software Development

22

22

Scrum Values

Commitment

- Each person is **committed** to the project's goals.
- That level of commitment can be achieved when the team has the authority to make decisions in order to meet those goals, and everyone has a say in how the project is planned and executed.

Agile Software Development

23

23

Scrum Values

Respect

- Team members **respect** each other.
- When team members have mutual respect, they're able to trust each other to do a good job with the work they've taken on.

Agile Software Development

24

24

Scrum Values

Focus

- Everyone is **focused** on the work.
- When every team member is focused on the sprint goals and given the freedom to do whatever work is needed to meet those goals, the whole team is able to organize itself and easily redirect whenever a change is needed.

Agile Software Development

25

25

Scrum Values

Openness

- The teams value **openness**.
- When you're working on a Scrum team, everyone else on the team should always be aware of what you're working on and how it moves the project toward its current goals.
- That's why the practices in the basic Scrum pattern are aimed at encouraging openness among the team members.

Agile Software Development

26

26

Scrum Values

Courage

- Team members have the **courage** to stand up for the project.
- When you choose openness over opaqueness, you make the team stronger, rather than making yourself stronger at the expense of the team.
- It takes courage to do that, but when you do, you end up with a better product and a better work environment.

Agile Software Development

27

27

User Stories

- Agile teams use **user stories** as an effective tool to capture user needs.
- A user story is a simple, brief description of how a user interacts with the software.
- Typically, user stories are 1–4 sentences long.
- Many teams follow the guideline that a user story should fit on a 3×5 index card.

Agile Software Development

28

28

User Stories

- User stories are often written in a structured, fill-in-the-blanks format.

As a <type of user>, I want to <specific action I'm taking> so that <what I want to happen as a result>

- This format helps clarify the user's role, their goal, and the intended benefit.

Agile Software Development

29

29

User Story Example

Nominate a video for an achievement

As a returning user with a large friends list,
I want to nominate one friend's video
for an achievement
so that all of our mutual friends can vote
to give him a star.

Agile Software Development

30

30

User Story Example

- Clearly identifies **who the user is**: “a returning user with a large friends list.”
- Specifies **what the user wants to do**: “nominate one friend's video for an achievement.”
- Explains **why the user wants to do it**: “so that all of our mutual friends can vote to give him a star.”
- Includes a **descriptive title**: “Nominate a video for an achievement” – makes it easy for the team to reference.

Agile Software Development

31

31

User Stories

- User stories help teams manage their backlog easily.
- Agile teams often use backlogs made up almost entirely of user stories.
- User stories are simple and written from the user's perspective.
- This simplicity makes it easy for the Product Owner to review stories with users and stakeholders.

Agile Software Development

32

32

User Stories

- Stories can be prioritized based on value.
- Since user stories are small, they are easy to add, remove, or reorder.
- During sprint planning, the team selects stories from the backlog to deliver in the sprint.
- Team discusses user stories with the Product Owner to ensure clarity and define completion criteria.

Agile Software Development

33

33

User Stories

- Stories are broken down into smaller tasks.
- Tasks are estimated for time and effort.
- Tasks are added to the “To Do” column on the task board.
- Team members pick the most valuable available task when ready to work.
- The selected task is moved to the “In Progress” column and assigned to the team member.

Agile Software Development

34

34

Conditions of Satisfaction

- **Conditions of satisfaction** help developers understand what the completed software should look like.
- These conditions also help measure progress toward completion.
- Conditions of satisfaction are also known as “**acceptance criteria**” by some teams.

Agile Software Development

35

35

Conditions of Satisfaction

- Conditions of satisfaction are simple but serve an important and complex purpose.
- They are defined for each **user story** to clarify what the user must be able to do once the story is implemented.
- These conditions are usually concise enough to fit on the **back of a 3×5 index card** with the user story.
- **Product Owner** typically helps write or reviews the conditions of satisfaction.

Agile Software Development

36

36

Conditions of Satisfaction

Nominate a video for an achievement

Conditions of satisfaction

- * A user can nominate a video for an achievement
- * A user's friend is notified when his video gets an achievement
- * A user can see all of the videos his friends have nominated
- * A video with an achievement is displayed with a star next to it

Agile Software Development

37

37

Conditions of Satisfaction

- Conditions of satisfaction helps developers avoid declaring a feature as complete early.
- It provide a clear, concrete definition of Done.
- They help both the team and Product Owner determine when a user story is complete.
- A user story is considered "Done" only when all related work—**understanding, building, testing, and deploying**—is finished.

Agile Software Development

38

38

Conditions of Satisfaction

- The story is done when a user can execute all conditions in the software as they would be shown in the sprint review.
- Until that point, the responsible team member remains focused solely on that user story.

Agile Software Development

39

39

Story Points and Velocity

- Sprint planning involves the whole team deciding how much work can be done in the sprint.
- The goal is to set a target for the working software to be delivered at the end of the sprint.
- There are various ways teams estimate sprint workload.
- A common and effective technique in Scrum is using **story points**.

Agile Software Development

40

40

Story Points and Velocity

- Story points represent the effort required to complete a user story.
- Teams assign points by comparing the current story to similar past stories they've completed.
- No strict rule exists for how many points to assign to a user story.
- Common schemes include using 1 to 5 points, 1 to 10 points, Fibonacci sequence, or exponential numbers.

Agile Software Development

41

41

Story Points and Velocity

- The key is to use a consistent point system from sprint to sprint.
- The chosen system should be one the entire team is comfortable with.
- A story with the same point value should require a similar amount of effort.
- By assigning points to stories over time, teams can track how many points they complete per sprint.
- This measurement is known as **velocity**.

Agile Software Development

42

42

Story Points and Velocity

- If a team completes on average, say, 25 points' worth of stories in a sprint, then their project velocity is 25 points per sprint.
- Teams usually maintain a consistent velocity across multiple sprints.
- It is difficult to predict a team's velocity before they start working.
- Velocity data from past sprints can be used to plan future sprints.

Agile Software Development

43

43

Story Points and Velocity

- Past performance is not a guarantee of future returns.
- The same principle applies to story points in Scrum.
- Even if a team burned 32 points last sprint and 29 the sprint before, it doesn't guarantee similar output in the next sprint.

Agile Software Development

44

44

Story Points and Velocity

- Various unpredictable factors can affect a sprint, such as:
 - Misjudged story complexity
 - Unexpected technical issues
 - Team members taking vacation
 - Team members quitting
 - Other life events
- Despite variability, story points and project velocity tend to be reliable over time.

Agile Software Development

45

45

Sprint Planning with Story Points

- Begin sprint planning with the most valuable user stories from the product backlog.
- Select a small story as a baseline and compare it to a similar story from a previous sprint to assign story points.
- Discuss the estimate with the team:
 - Increase points if there are additional challenges or work.
 - Decrease points if simplifications or reusable elements are identified.

Agile Software Development

46

46

Sprint Planning with Story Points

- Continue estimating stories until the sprint capacity (in story points) is reached.
- Avoid overloading the sprint backlog. Leaving some room in the backlog is acceptable.
- Do not exceed the team's past average velocity.
- Stick to the team's proven capacity to ensure reliable delivery.

Agile Software Development

47

47

Sprint Planning with Story Points

- Estimating story points for the first time can be challenging due to lack of history.
- Initially, make educated guesses based on perceived effort.
- Pick a mid-level story and assign it **3 points**.
- Identify the largest valuable story and assign it **5 points**.
- Find the smallest story and assign it **1 point**.

Agile Software Development

48

48

Sprint Planning with Story Points

- Use these initial estimates to gauge other stories and plan your sprint.
- After completing two sprints, you'll have:
 - A set of reference stories for comparison.
 - A better understanding of your team's average velocity.

Agile Software Development

49

49

Burndown Charts

- A **burndown chart** visually tracks sprint progress.
- It allows anyone to quickly assess how the sprint is progressing.
- Progress is compared with the team's past velocity.
- This chart can be built using story points or other measurements such as hours or days.

Agile Software Development

50

50

Burndown Charts

- Start with an empty line chart.
- **X-axis:** Dates from the first to the last day of the sprint.
- **Y-axis:** Story points, ranging from 0 to 20% more than the total sprint backlog.
- Plot the **first dot** at day 0, representing the total number of sprint points.
- Draw a **straight line (guideline)** from this first point to the last day, ending at 0 points (indicating sprint completion).

Agile Software Development

51

51

Burndown Charts

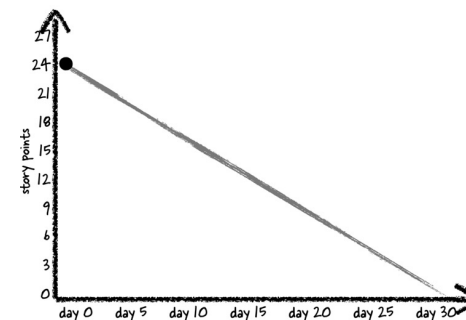


Figure: Burndown chart when the sprint starts

Agile Software Development

52

52

Burndown Charts

- Once the first user story is completed and moved to the “Done” column, mark the next dot on the burndown chart.
- The dot represents the number of points remaining in the sprint on that day.
- Continue updating the chart each day as more stories are finished.
- This process tracks the burn-down of points from the backlog over the sprint duration.

Agile Software Development

53

53

Burndown Charts

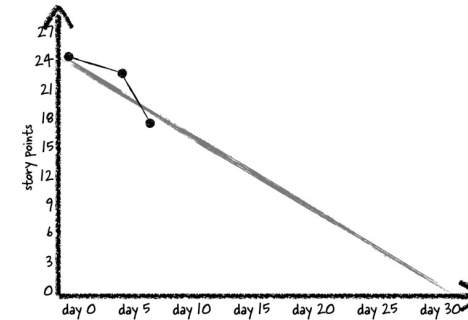


Figure: After 2 stories worth 7 points burned off

Agile Software Development

54

54

Burndown Charts

- During a Daily Scrum, new work may need to be added to the sprint.
 - If the team is burning through points faster than expected and will finish early.
 - New important support tasks might arise and be agreed upon by the team and product owner to add to the sprint.
- Initially, it may be unclear how much existing work needs to be removed to balance the sprint.

Agile Software Development

55

55

Burndown Charts

- When adding new work cards to the task board, schedule a follow-up meeting with the team.
- In the follow-up, estimate story points for each new card and update the sprint chart.
- Draw an extra line on the chart to indicate where new points were added.
- Writing notes on the chart is encouraged to keep track of changes.

Agile Software Development

56

56

Burndown Charts

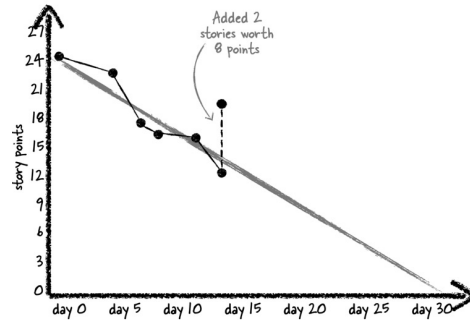


Figure: After stories added halfway through the sprint

Agile Software Development

57

57

Burndown Charts

- As the sprint nears its end, more points are completed and removed from the chart.
- Monitor for any gap between the guideline (planned progress) and the actual burn-down.
- A gap may indicate too many points remain in the sprint.
- If so, consider removing a user story to stay on track.

Agile Software Development

58

58

Burndown Charts

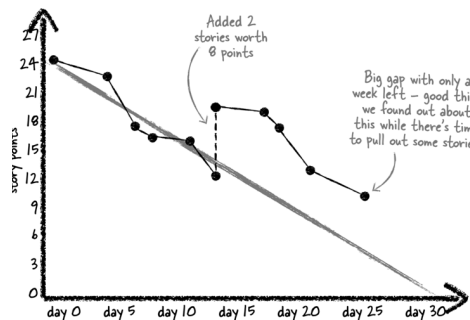


Figure: Visible gap between the burndown chart and the guideline

Agile Software Development

59

59

Burndown Charts

- Many software tools exist to manage backlogs, stories, story points, and automatically generate burndown charts.
- Despite this, many teams prefer to draw burndown charts manually.
- Manual charts are often kept on the same wall or whiteboard as the task board.
- This setup allows everyone to easily see the project's progress at any time.

Agile Software Development

60

60

Tasks and Task Board

- A Sprint Planning Meeting is necessary to answer two key questions:
 1. What will the team deliver in this sprint?
 2. How will the team do the work?
- Teams commonly use **story points and velocity** to determine the sprint's deliverables (answering the first question).
- Second half of sprint planning focuses on **how** the team will execute the work.

Agile Software Development

61

61

Tasks and Task Board

- Scrum teams commonly plan work by creating **task** cards.
- These cards represent individual development tasks.
- These tasks can be anything that the team actually does.
 - Write code, create design and architecture, build tests, install operating systems, design and build databases, deploy software to production servers, run usability tests, etc.

Agile Software Development

62

62

Tasks and Task Board

- Team holds the second sprint planning meeting.
- Scrum Master initiates the discussion starting with the first user story.
- Team collaborates to determine the necessary steps to build the story.
- They break down the story into individual tasks, each expected to take no more than one day.
- Each task is written on a separate card.

Agile Software Development

63

63

Tasks and Task Board

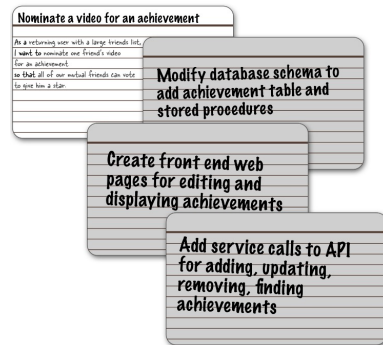
- Some teams use different colored cards to distinguish between story and task cards.
- Task cards are grouped with their corresponding story card.
- This process continues until all user stories are planned.
- If the sprint planning timebox expires before all of the stories are broken down, each unplanned story gets a single task card for a task to plan out the story.

Agile Software Development

64

64

Tasks and Task Board



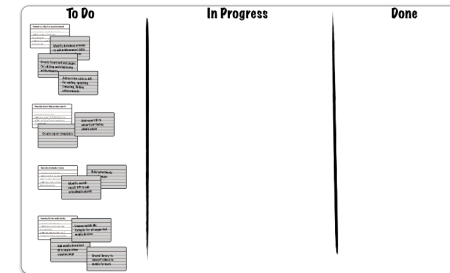
Agile Software Development

65

65

Tasks and Task Board

- Stories and their tasks are grouped together and added to the “To Do” column of the **task board**.



Agile Software Development

66

66

Tasks and Task Board

- When a team member completes a task, they move its card to the “Done” column.
- They then choose a new task from the “To Do” column.
- They write their name on the new task card and move it to the “In Progress” column.
- If the new task is part of a story still in the “To Do” column, they also move the story to “In Progress”.

Agile Software Development

67

67

Tasks and Task Board

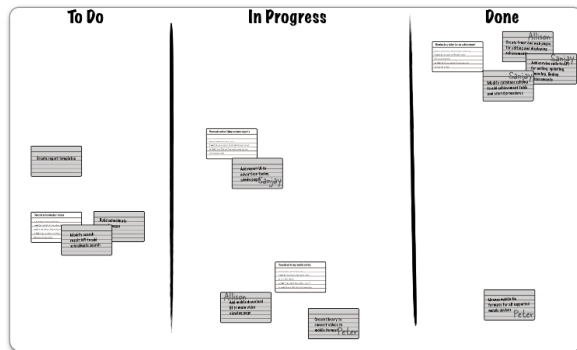
- When a team member finishes the final task for a story, they:
 - Remove the story card from the task board.
 - Verify that all **conditions of satisfaction** are completed.
 - Move the story to the “**Done**” column with all its tasks.
- However, the story is not fully done until the Product Owner accepts it as potentially shippable.

Agile Software Development

68

68

Tasks and Task Board



Agile Software Development

69

69

Tasks and Task Board

- If any **condition of satisfaction** is not met:
 - Story is moved back to the “**In Progress**” column.
 - New tasks are added to the “**To Do**” column to complete the remaining work.
- A sprint ends when the timebox expires, regardless of task completion.
- Story and task cards left in “To Do” or “In Progress” are returned to the product backlog.

Agile Software Development

70

70

Tasks and Task Board

- These items can be re-prioritized in the next sprint planning session.
- The team may reduce the story point estimate based on remaining tasks.
- No credit is given for a story unless both the story card and all associated task cards are in the “Done” column.

Agile Software Development

71

71